Modular

Value Ownership, Lifetimes, and Origins: Mojo's Memory Safety Model



Chris Lattner

Modular, a Qualcomm Company

LLVM dev meeting 2026

Representing the Mojo team and community

KEY TAKEAWAY:

Mojo is a memory safe language.

It prevents use-after free, does not require exclusive mutabilty, and hides lifetime management complexity until you really need it.

Agenda

01 (Very quick) introduction into the Mojo parameter syntax

02 What is 'memory safety'

03 How Rust does it

04 How Mojo is different: origins vs lifetimes

05 What origins give to the programmer

Important Mojo lingo to know

Parameter means
“compile time value”

Argument means
“runtime value”

Mojo parameter syntax

A “parameterized” function

```
def example[dtype: DType, width: Int](x: SIMD[dtype, width]):
```

compile-time parameters | run-time argument

C++ equivalent

```
template<DType dtype, int width> void example(SIMD<dtype, width> const& x)
```

- Mojo parameters are more general (than C++): types and functions can be parametrized by *any type or value* that Mojo has.
- Any expression *evaluated at compile time* can be used to *instantiate* a parametric declaration.

Memory safety

Every access to memory goes through a reference derived from the object being accessed, within that object's bounds, and only while that object is alive.

> A language is **memory safe** if programs in its safe subset cannot violate this, no matter what they do.

HOW RUST DOES MEMORY SAFETY

Rust tracks the lifetime: a region of code

```
let xs: Vec<i64> = vec![1, 2, 3];
let s: &'a [i64] = &xs[..];
println!("{}", s[0]);
println!("{}", xs.len());
```

s ⟶ xs live over lines 2–3

Lifetimes are declared

The programmer names regions with annotations like `'a`.

The borrow checker verifies

Every reference must stay inside the region its owner is alive.

Familiar, and powerful

Rust brought memory safety without GC to the mainstream.

Well-known limitations:

- No overlapping mutable references
- Lifetime annotations spread through signatures and structs

Mojo difference: Lifetime is computed, not declared

Rust

`lifetime 'a written in the type` → `checker verifies references stay within it`

The lifetime is something you declare.

Mojo

`origin 'o: which objects a reference might point to` → `compiler computes each object's lifetime` → `inserts destructors after last use`

The reference only says what it points to; the lifetime is maintained across the last reference.

Key concepts in Mojo's ownership system

01

Each memory object has an origin

A symbolic ID for the value.

```
# origin: 'list
var list : List[Int]
```

→

02

References propagate it

The ID propagates through generic code that uses references.

```
# origin = 'list
ref r = list
```

→

03

Lifetimes are computed

Mojo combines this information to find each value's lifetime.

```
# last use of 'list
use(r)
```

→

04

Destructors & diagnostics

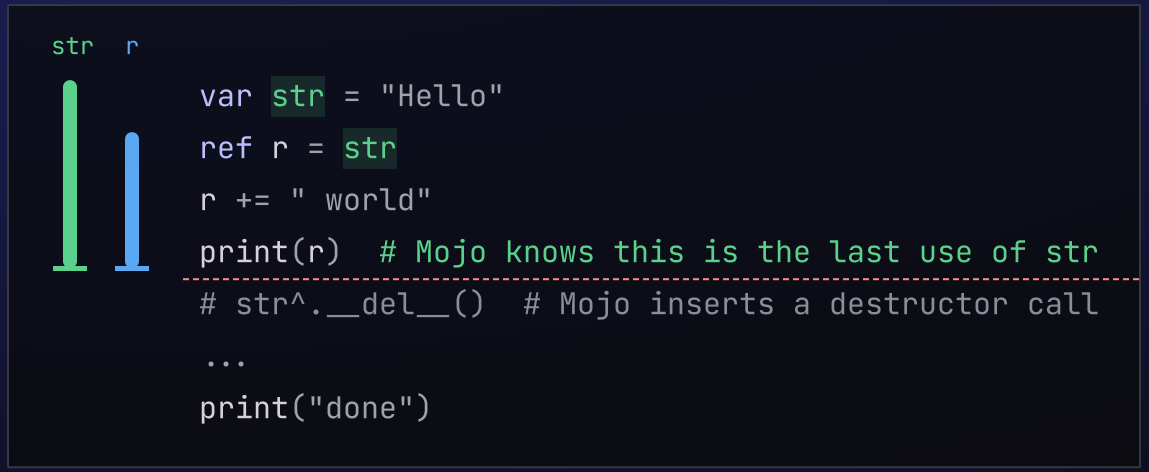
Mojo inserts destructor calls and diagnoses problems.

```
use(r)
list.__del__()
```

Origins are how Mojo **knows what a reference returned by a function might point to**, and how it **enforces exclusivity** and how it catches incorrect mutable reference aliasing.

Lifetimes are computed from origin access sets

Values are destroyed ASAP after the last reference is gone



r keeps str alive

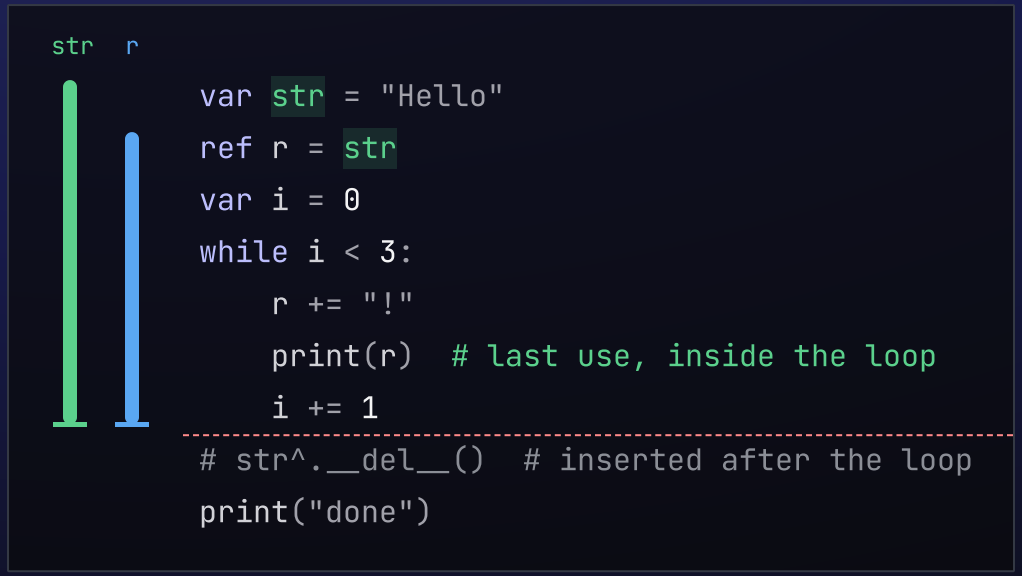
Its origin is `str`, so `str` lives until `r`'s last use

Destroyed at the dashed line

Right after the last use, not at the end of the scope

Memory safety implemented via dataflow analysis

Compute lifetimes → check for violations → insert destructors



r stays live around the loop

The loop's back edge reaches `r`'s use again, so `str` must live through every iteration

Destroyed after the loop

Not after the textually last use; liveness is computed over the control-flow graph

Two dataflow passes in `Mojo/lib/LowerLIT/CheckLifetimes.cpp`

Forward pass finds uninitialized values, backward pass inserts destructors after last use.

The analysis is intra-procedural. The programmer writes transfer functions for each function call (next slide).

Origin parameters → origin transfer functions

As part of function signature, code author writes a symbolic transfer function from input origins to result origins.

```
def take_and_return_ref[o: Origin](
    ref [o] arg: Int
) -> ref [o] Int:
    return arg
```

```
var my_int = 17

# This returns a reference to my_int.
ref my_ref = take_and_return_ref(my_int)

# Both names are valid, Mojo knows my_ref refers to my_int
print(my_ref)
print(my_int)
```

OR, MORE CONCISELY

```
def take_and_return_ref(
    ref arg: Int
) -> ref [arg] Int:
    return arg
```

Origins name the owner

The origin `o` is a comptime parameter: the returned reference has the same owner as `arg`.

Type checking is modular and fast

Callers are checked against the signature alone; the compiler never needs to see the function's body.

Benefits of ASAP destruction

Values die at their last use, so live ranges are as short as possible.

01

Smaller async capture sets

A coroutine frame only holds values still live across an `await`.

```
await
```

02

Tail calls work

Nothing is left to destroy after the call, so it can be a true tail call.

```
return f(x)
```

03

Expensive resources freed early

Handles to GPU memory and other scarce resources are released when you're done, not at end of scope.

```
DeviceBuffer
```

04

Copy → move optimization

A copy that is the last use of a value reliably turns into a move.

```
x = y  # copy
→
x = y^ # move
```

RAII doesn't fit ASAP destruction, since values die at last use, not end of scope. Mojo uses Python-style context managers (`with` blocks) and **linear types** for things that need explicit cleanup.

Mojo argument conventions sugar standard cases

WHAT YOU WRITE

```
def append_twice(mut dst: String, src: String):
    dst += src
    dst += src
```

≡

LONG HAND, WITH REF

```
def append_twice[o1: MutOrigin, o2: ImmOrigin](
    ref [o1] dst: String, ref [o2] src: String
):
    dst += src
    dst += src
```

`mut dst: String`	≡	`ref [_: MutOrigin] dst: String`
`src: String`	≡	`ref [_: ImmOrigin] src: String`

Well-chosen defaults mean many Mojo programmers never have to think about references, while getting the benefits of strong memory safety and no copies or extraneous moves.

Origins can track a set of locations

```
def longest(ref a: String, ref b: String) → ref [a, b] String:
    if a.byte_length() > b.byte_length():
        return a
    return b

def main():
    var cat = String("cat")
    var horse = String("horse")
    ref winner = longest(cat, horse)   # winner's origin: {cat, horse}
    print(winner)                      # cat and horse are destroyed here
```

Origins track mutability, and mutability is parametric

MOJO: ONE FUNCTION

```
longest(cat, horse) += "radish"
print(horse)  # "horseradish"
```

C++/RUST: DUPLICATE IMPLEMENTATIONS

```
const string& longest(const string& a, const string& b);
string&       longest(string& a, string& b);
```

How? Mutability is a parameter of the origin

ORIGIN IS A STRUCT WITH A MUT PARAMETER

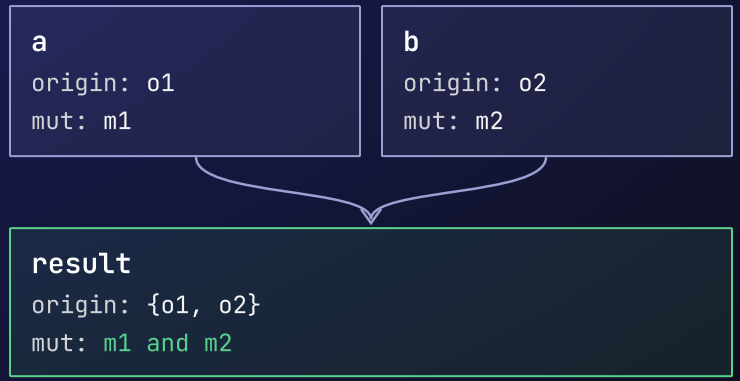
```
struct Origin[mut: Bool]:
    ...
```

LONGEST TAKES TWO ORIGIN PARAMETERS

```
def longest[o1: Origin, o2: Origin](
    ref [o1] a: String, ref [o2] b: String
) → ref [o1, o2] String
```

EACH ORIGIN HAS ITS OWN MUTABILITY

```
def longest[
    m1: Bool, o1: Origin[m1],
    m2: Bool, o2: Origin[m2],
](ref [o1] a: String, ref [o2] b: String)
    → ref [o1, o2] String       # mut = m1 and m2
```



	b mut	b immut
a mut	mut	immut
a immut	immut	immut

A reference is mutable only if **all** of its origins are mutable.

Origin parameters work with structs and other types

```
struct Span[T: AnyType, origin: Origin]:
    var _data: Pointer[Self.T, Self.origin]
    var _len: Int

    def __getitem__(self, idx: Int) -> ref [Self.origin] Self.T:
        debug_assert(0 <= idx < self._len, "index out of bounds")
        return self._data[idx]
```

Span struct in the Mojo standard library — the struct's origin parameter flows into its field's pointer type.

Mutable ref exclusivity is checked at program points

This enables flexible overlapping references

MOJO ALLOWS OVERLAPPING REFERENCES

```
var str: String = "foo"
ref r = str   # r is a mutable ref, doesn't disable str
print(str)    # reading str while r is live is fine
r += "bar"
print(str)       # "foobar"
```

Rust rejects this (E0502). The **mutable** element borrow locks the whole string.

MOJO REQUIRES ALIASING XOR MUTABILITY AT CALL SITES

```
var str: String = "foo"
ref r = str
...
swap(r, str)
# error: cannot pass mutable references to the same value twice
```

Interior origin: a reference into a container

Interior origins are flow sensitive, invalidated when the value they are based on is mutated

```
var xs: List[Int] = [1, 2, 3]
ref e = xs[0]      # e tracks that it points into xs
xs.append(4)       # mutates the xs, invalidates e

# Using the e reference would be potentially dangling: unsafe
e += 41
```

Mojo uses **flow-sensitive analysis** to invalidate the reference - the same machinery it uses to track memory as uninitialized when the value is moved out of.

```
error: use of invalidated interior reference 'xs["element"]'
    e += 41
    ^
note: origin was invalidated here
    xs.append(4)
              ^
```

What a Mojo programmer gets from Origins

The benefits of Rust lifetimes	Memory safety, no garbage collector, systems-level control, and an escape to unsafe APIs when needed, with fewer annotations.
Accepts code Rust rejects	Overlapping references, more usability for references into containers.
Better ergonomics for the common case	Mojo programmers don't need to think about references, until they define an API that returns one.
Parametric mutability	Don't duplicate functions: single accessor works for mutable and immutable uses
ASAP destruction of values	Smaller capture sets, more tail calls, copy → move optimization. Linear types that compose with the rest of the type system.

No new sublanguage: origins are just parameters, so inference, constraints, and generics all apply.

Modular

Thank you

github.com/modular/modular
modular.com

